

# Contents

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| <b>Unstructured Data Tagging &amp; Reporting — Handover Document</b> | <b>1</b> |
| 1. Why we are doing this . . . . .                                   | 1        |
| 2. What we are doing . . . . .                                       | 1        |
| 3. How we are doing it . . . . .                                     | 2        |
| 4. Why we sign the scripts . . . . .                                 | 3        |
| 5. Why the agent must be started in detached mode . . . . .          | 6        |
| 6. Technical walkthrough — UDR-Tagging-Parallel.ps1 . . . . .        | 6        |
| 7. Technical walkthrough — update_pdf_properties.py . . . . .        | 11       |
| 8. The reporting side (supporting scripts) . . . . .                 | 12       |
| 9. Operational checklist for a new owner . . . . .                   | 15       |

## Unstructured Data Tagging & Reporting — Handover Document

**Prepared as a handover for anyone taking over or supporting the Unstructured Data Remediation (UDR) tagging and reporting process.**

*UDR — Unstructured Data Remediation — is the name of the project and the activity described in this document. This document explains *why* the process exists, *what* it does, *how* it is deployed, and provides a technical walkthrough of each script so a new owner can operate, troubleshoot, and safely modify it.*

---

### 1. Why we are doing this

The organisation holds large volumes of unstructured files on NAS / file shares. Some of these files:

- may contain **personal information**, and
- are **out of retention** — old enough that, under the organisation's retention rules, they should no longer be held.

Microsoft Purview retention policies can manage content in Microsoft 365 (SharePoint, OneDrive, Exchange), but they **cannot reach files sitting on NAS drives / traditional file shares**. That leaves a gap: potentially sensitive, over-retention files that no automated retention control can see or act on.

The purpose of this process is to **close that gap** by making those files discoverable and actionable so they can be **quarantined** (moved out of general access / flagged for deletion or review). We do this by writing custom metadata onto each file that indicates its age/retention status, so downstream tooling and reporting can identify which files are candidates for quarantine.

In short: **Unstructured Data Remediation tags on-premises unstructured files with age/retention markers that Purview-style controls otherwise couldn't apply, then reports on them so the out-of-retention, potentially-personal-data files can be dealt with.**

---

### 2. What we are doing

We run two scripts that write **custom document properties** into files:

| Script                   | Handles                                                                                          | Mechanism                                                                                               |
|--------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| UDR-Tagging-Parallel.ps1 | Microsoft Office files (Word, Excel, PowerPoint — both legacy binary and modern OpenXML formats) | PowerShell, via Office COM automation for legacy formats and direct OpenXML/OLE manipulation for others |
| update_pdf_properties.py | PDF files                                                                                        | Python (pypdf), called by the PowerShell script                                                         |

The properties written to each file are:

| Property             | Meaning                                                                                |
|----------------------|----------------------------------------------------------------------------------------|
| OriginalPath         | The full path the file was tagged at (so it can be traced back even if moved)          |
| LastAccessed18Months | True if the file has <b>not</b> been accessed in the last <b>540 days (~18 months)</b> |
| Created3Years        | True if the file was <b>created more than 1095 days (~3 years)</b> ago                 |

These are written as **text** values "True" / "False" (not booleans) specifically so Purview / downstream tooling reads them reliably.

A key design principle throughout: **tagging must be invisible to the business**. The scripts capture each file's original LastWriteTime, LastAccessTime, and read-only flag before touching it, and **restore them afterwards**, so that adding a property doesn't make a file look "recently modified" or reset its access date (which would defeat the whole retention-age logic).

### 3. How we are doing it

#### Infrastructure:

- Azure **Windows servers** built specifically for this work, with:
  - **Microsoft Office installed** (required for COM automation of legacy .doc / .xls / .ppt files), and
  - **Azure DevOps (ADO) self-hosted agents** installed.
  - **Python** installed (with pypdf) for the PDF path.
- The scripts are **stored on one server** (a single source of truth). A second server can execute the same script directly over a UNC path (e.g. \\server1\Temp\UDR-Tagging-Parallel.ps1), and the server used for a given run is chosen based on current activity/load.
- An **Azure DevOps pipeline** invokes the script for a designated server, passing in the target DrivePath (the share/folder to scan) and ScriptPath (where the script and its helper files live) as parameters.
- All tagging runs and report extracts execute under the dedicated service account **{{windows domain}}\UDRTagging**. This account is what the ADO agent runs as, and it must hold the file, log, COM/Office, and certificate-key permissions referenced throughout this document.
- **The process relies on {{windows domain}}\UDRTagging being a member of the Local Administrators group on the host server of the target drive**. This is required so the account can access **all** files on the drive (including folders whose ACLs would otherwise deny it), so that no in-scope files are silently missed during scanning and tagging. When a new drive/share is brought into scope, adding {{windows domain}}\UDRTagging to the local admin group of the server hosting that drive is a prerequisite step.

**Note — hosted agents will not work.** Microsoft-hosted ADO agents do not have Office installed, so COM automation would fail immediately. This is why **self-hosted agents on Office-equipped servers** are used.

**Scheduling which drive runs where and when.** With multiple servers and many drives to work through, a small Excel model (UDR\_Execution\_Scheduler.xlsx) plans the run order: each drive has a forecasted duration, each server can only run one drive at a time, and drives can be prioritised either by **location** or by **volume** (largest first). Per-server availability can be overridden (e.g. when a server is mid-job) so the plan re-simulates around real start times rather than a single shared start. This is a planning aid for deciding *which* drive goes to *which* server *next* — it doesn't drive the pipeline itself, but is the source for the DrivePath values fed into pipeline runs.

#### High-level flow of a run:

1. Pipeline triggers the PowerShell task on the chosen server.
  2. The agent (running in an interactive desktop session — see §5) executes UDR-Tagging-Parallel.ps1 with the target DrivePath.
  3. The script enumerates applicable files, classifies each, and dispatches them in parallel batches to the correct handler (OpenXML / legacy COM / PDF).
  4. Each file is tagged with the three properties, timestamps restored, and the outcome logged.
  5. Separately, reporting scripts scan the folder tree and load results into SQL Server for analysis (see §8).
- 

## 4. Why we sign the scripts

The execution server runs under PowerShell execution policy **AllSigned**, enforced by **Group Policy** (confirmed via `Get-ExecutionPolicy -List` showing `MachinePolicy = AllSigned`). Under **AllSigned**, **every** script must carry a valid Authenticode signature before it will run — there is no exception for trusted locations, and importantly, a script executed from a **UNC path** (our “store on one server, run from another” setup) is treated as remote and blocked without a signature.

The symptom when this isn't satisfied is:

*“File cannot be loaded. The file is not digitally signed. You cannot run this script on the current system.”*

Because **AllSigned** is set by GPO, it **cannot** (and should not) be overridden locally — the domain refreshes and reverts any local change. Rather than fight the policy, we **work with it** by self-signing.

#### How signing is set up:

- A **self-signed code-signing certificate** (CN=UDR Tagging Script Signing) was created once, in the **LocalMachine** store (not **CurrentUser**), so it is available to *all* accounts — including the ADO agent's service account, not just an interactive admin.
- The **public certificate** is imported into `LocalMachine\TrustedPublisher` and `LocalMachine\Root` on every server that *runs* the script, so the signature is trusted without prompting.
- The certificate must also be trusted on the **signing** server itself (into its own **Root**), otherwise signing fails with “a certificate chain processed but terminated in a root certificate which is not trusted.”
- (*Hypothetical / future only*) If the private key were ever exported to a **separate** signing server and the agent there ran as a low-privileged service account, that account would need **read access to the certificate's private key** to sign — see the note on current state below.

**Current state — self-signed, single certificate.** The certificate is presently **self-signed** and the signing is done on the server where it was created, so there is **no separate private-key (.pfx) distribution in play today**. The private-key export/import and key-ACL steps below are included only for completeness — they describe what *would* be needed if signing were ever moved to, or duplicated on, another server. At this stage they are **hypothetical** and can be skipped; the only

setup actually required right now is trusting the **public** certificate on each server that runs the scripts.

### Importing the certificate into a server's certificate store

This is a **one-time** setup per server (the signature itself is re-applied often, but the trust setup is not). The distinction that matters is *what* you import on *which* server:

- A server that only **runs** the signed scripts needs the **public certificate** trusted in its trust stores. (*This is the only case that applies today.*)
- (*Hypothetical / future only*) A server that also **signs** the scripts would need the **private key** in its LocalMachine\My store, and the public certificate trusted in its own Root store. There is no such separate signing server at present.

The public certificate has already been created and is held on the script share as **\\server1\temp\UDR-CodeSigning.cer** (alongside the scripts), so on a run-only server you can import straight from there without re-exporting — skip to Step 3. The export commands in Step 1 are only needed if you have to reissue the certificate or produce the private-key (.pfx) file for a new signing server.

*Step 1 (only if reissuing / setting up a new signing server) — export the certificate from the signing server.*

Public certificate only (for servers that run the scripts):

```
$cert = Get-ChildItem Cert:\LocalMachine\My -CodeSigningCert |  
    Where-Object { $_.Subject -eq "CN=UDR Tagging Script Signing" } |  
    Select-Object -First 1  
  
Export-Certificate -Cert $cert -FilePath "\\server1\temp\UDR-CodeSigning.cer"
```

Private key (only if another server also needs to *sign* — protect this file, it contains the key):

```
$pwd = Read-Host "PFX password" -AsSecureString  
Export-PfxCertificate -Cert $cert -FilePath "\\server1\temp\UDR-CodeSigning.pfx"  
    -Password $pwd
```

*Step 2 — on the target server, run PowerShell as Administrator* (writing to LocalMachine stores requires elevation). The .cer can be imported directly from \\server1\temp; no local copy is needed.

*Step 3 — import on the target server.*

On a server that **runs** the scripts, import the public certificate into **both** TrustedPublisher (so the signer is trusted) and Root (so the self-signed chain validates):

```
Import-Certificate -FilePath "\\server1\temp\UDR-CodeSigning.cer" -  
    -CertStoreLocation Cert:\LocalMachine\TrustedPublisher  
  
Import-Certificate -FilePath "\\server1\temp\UDR-CodeSigning.cer" -  
    -CertStoreLocation Cert:\LocalMachine\Root
```

(*Hypothetical / future only — does not apply today.*) If signing were ever moved to or duplicated on a **separate** server, that server would additionally need the private key imported into LocalMachine\My. The .pfx would only be produced at that point (it is **not** created or kept on the share today, since it contains the private key):

```
$pwd = Read-Host "PFX password" -AsSecureString  
Import-PfxCertificate -FilePath "\\server1\temp\UDR-CodeSigning.pfx" -  
    -CertStoreLocation Cert:\LocalMachine\My -Password $pwd
```

The **signing** server must *also* trust the certificate in its own Root store (the two Import-Certificate commands above), otherwise signing fails with “a certificate chain processed

*but terminated in a root certificate which is not trusted* — signing validates the chain even on the machine doing the signing.

Step 4 — verify the import.

```
# Confirm it's trusted as a publisher and as a root
Get-ChildItem Cert:\LocalMachine\TrustedPublisher | Where-Object { $_.Subject -like
  ↳ "*UDR Tagging Script Signing*" }
Get-ChildItem Cert:\LocalMachine\Root | Where-Object { $_.Subject -like "*UDR
  ↳ Tagging Script Signing*" }

# On a signing server, confirm the private key is present (HasPrivateKey should be True)
Get-ChildItem Cert:\LocalMachine\My -CodeSigningCert |
  Where-Object { $_.Subject -eq "CN=UDR Tagging Script Signing" } |
  Select-Object Subject, Thumbprint, HasPrivateKey, NotAfter
```

Step 5 (hypothetical / future — only if a separate signing server with a low-privileged agent account is ever introduced) — grant the agent read access to the private key, so it can sign during the pipeline. By default only SYSTEM and Administrators can read it; if the agent runs as SYSTEM or a local admin this step isn't needed. **This does not apply to the current single-server, self-signed setup:**

```
$cert = Get-ChildItem Cert:\LocalMachine\My -CodeSigningCert |
  Where-Object { $_.Subject -eq "CN=UDR Tagging Script Signing" } |
  Select-Object -First 1

$keyPath = "$($env:ProgramData)\Microsoft\Crypto\RSA\MachineKeys"
$keyName = [System.Security.Cryptography.X509Certificates.RSACertificateExtensions]::
  ↳ GetRSAPrivateKey($cert).Key.UniqueName
$keyFile = Get-Item "$keyPath\$keyName"

$acl = Get-Acl $keyFile.FullName
$rule = New-Object System.Security.AccessControl.FileSystemAccessRule("{windows
  ↳ domain}}\UDRTagging", "Read", "Allow")
$acl.AddAccessRule($rule)
Set-Acl $keyFile.FullName $acl
```

*({{windows domain}}\UDRTagging is the account used to run the tagging and report extracts — this is the account shown as “Log On As” for the DevOps agent service in services.msc.)*

**Why it has to be re-signed regularly:** an Authenticode signature is a hash of the file's exact contents at signing time. **Any** edit to the script invalidates the signature. Because the script has changed frequently, re-signing is built in as a **pipeline step** (the Sign-\*.ps1 scripts) that runs immediately after the file is written/updated, so the file is always signed before any run attempts to execute it.

The signing scripts (Sign-UDRTagging-Script.ps1, Sign-UDRFolder-Script.ps1, Sign-UDRCatalogue-Script.ps1, Sign-UDR-FolderReport-Script.ps1, Sign-Detached-Script.ps1) each:

1. Look up the code-signing certificate in Cert:\LocalMachine\My by **thumbprint** (using Select-Object -First 1 so a single certificate object is returned, not an array — otherwise Set-AuthenticodeSignature throws a type error).
2. Call Set-AuthenticodeSignature on the target script.
3. **Check \$result.Status -ne "Valid"** and fail the step if signing didn't succeed — Set-AuthenticodeSignature returns a status object rather than throwing, so without this check a failed signing would silently proceed and leave the script unsigned.

**Certificate expiry:** the certificate was created with a multi-year validity (≈5 years from July 2026). Diarise its renewal — once it expires, signing fails and every run is blocked until a new certificate is issued and trusted on all servers.

---

## 5. Why the agent must be started in detached mode

**The problem we hit:** the ADO pipeline would intermittently fail with “*the agent is not contactable.*” Root-cause tracing showed that Agent.Listener.exe was running as a **direct child of the interactive PowerShell console** that had been used to start it. As a consequence, anything that closed or disturbed that console — closing the window, Ctrl+C, an RDP session disconnect affecting the window, or the shell crashing — would **silently kill the agent**, with no crash log and no OS-level event. That matched the “not contactable” failures exactly.

**Why we can’t just run the agent as a normal background service:** the legacy Office COM automation (.doc / .xls / .ppt) **requires an interactive desktop session** to work. Running the agent in Session 0 / as a plain service breaks COM. So the agent genuinely needs to live in an interactive session — it just must not be *tied to one specific console window*.

**The fix — start-agent-detached.ps1:** this launches the agent’s run.cmd via Start-Process, which breaks the parent/child relationship. The agent still runs **within the interactive desktop session** (so COM still works) but is **no longer a child** of the console that launched it, so closing that console or disconnecting RDP no longer kills it. The script then verifies detachment by walking the process tree (Get-CimInstance Win32\_Process) and confirming the agent’s parent is **not** the launching shell’s PID, logging the result for audit.

**Operational rule of thumb:** always start/restart the agent using start-agent-detached.ps1 from an interactive session — **never** by running run.cmd directly in a foreground console, which reintroduces the exact fragility this fixes.

A secondary suspect investigated at the same time was **Office COM zombie/handle leakage** (orphaned WINWORD/EXCEL/POWERPNT processes accumulating over many runs, potentially exhausting memory or the per-session GDI handle limit). The tagging script mitigates this directly with an external kill-monitor and aggressive COM cleanup (see §6). If “not contactable” recurs *after* detachment is confirmed, check for orphaned Office processes and trend handle/memory counts.

---

## 6. Technical walkthrough — UDR-Tagging-Parallel.ps1

This is the core script. It takes two mandatory parameters:

- -DrivePath — the root folder/share to scan and tag.
- -ScriptPath — the working location holding the script, the Python helper, and where state/logs are written.

### 6.1 Overall shape

The script is one large file that (a) defines many helper functions, then (b) at the very bottom calls Invoke-ExecuteTaggingSafely, which starts the Office kill-monitor and then runs Execute\_Tagging. Execute\_Tagging builds the file list and hands it to Update-FileAgeProperties, which sorts files into three queues (OpenXML / legacy-COM / PDF) and dispatches them to the matching parallel batch processor.

The processing pipeline, end to end:

```
Invoke-ExecuteTaggingSafely
├─ Start-KillProcessMonitor (external watchdog for stuck Office processes)
└─ Execute_Tagging
```

- └ Get-ApplicableFiles (find old, in-scope files)
- └ Update-FileAgeProperties (queue + dispatch in batches)
  - └ Process-OpenXmlBatch (.docx/.xlsx/.pptx etc. – no Office needed)
  - └ Process-ComBatch (.doc/.xls/.ppt – Office COM)
  - └ Process-PdfBatch (.pdf – calls Python)

## 6.2 Function-by-function summary

### Discovery & orchestration

| Function                 | Purpose                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Execute_Tagging          | Entry point for a run. Resolves the target folder, sets up the per-run state directory (FilesToScan.txt, FilesScanned.txt, FilesSkipped.txt), and supports <b>resume</b> : on a re-run it skips files already scanned or skipped, so an interrupted run continues rather than restarting. Calls Update-FileAgeProperties, then archives the run's file lists with a timestamp.                  |
| Get-ApplicableFiles      | Recursively walks DrivePath and returns files that are <b>in scope</b> : supported extensions (.doc / .docx / .docm / .xls / .xlsx / .xlsb / .ppt / .pptx / .pptm / .pdf), <b>last accessed &gt; 540 days ago</b> , <b>created &gt; 1095 days ago</b> , non-zero size, and not a ~ temp/lock file. Logs every folder visited and any access errors, and keeps going past folders it can't read. |
| Update-FileAgeProperties | The dispatcher. Iterates the file list, skips missing/locked files, classifies each file by format, and pushes it onto the OpenXML, COM, or PDF queue. When a queue reaches its <b>batch trigger</b> threshold it dispatches that batch, waits for it, and drains it. Periodically nudges garbage collection to reclaim COM/runspace memory across a long run.                                  |
| Wait-AndCollectJobs      | Shared helper that waits on every job in a dispatched batch (with a per-job timeout), collects output, disposes each runspace pipe, and — crucially — <b>closes and disposes the runspace pool itself</b> , which is what actually frees the batch's threads and memory before the next batch starts.                                                                                           |

### Format detection & protection checks (used to route files and to *skip* anything we shouldn't touch)

| Function         | Purpose                                                                                                                                                                                                                 |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Get-OfficeFormat | Reads the first 8 bytes of a file to decide if it's OpenXML (ZIP signature PK), legacy BinaryOLE (OLE signature D0 CF 11 E0...), or an OLE-wrapped <b>encrypted</b> OpenXML file — routing each to the correct handler. |

| Function                                     | Purpose                                                                                                                                                                                                                                            |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Test-OfficeEncrypted                         | Determines whether an OLE/OOXML file is encrypted (password-to-open) by looking for EncryptedPackage / EncryptionInfo streams. Encrypted files are skipped (we can't and shouldn't open them).                                                     |
| Test-LegacyOfficeProtection                  | Detects <b>password-to-open</b> and <b>password-to-modify</b> in legacy binary .doc/.xls/.ppt <b>without opening them in Office</b> , by parsing the OLE compound-file structure in memory. Prevents the script from hanging on a password prompt. |
| Read-OleStream                               | Low-level helper for the above: reads a named OLE stream from a pre-loaded byte array by walking the FAT sector chain, avoiding repeated network round-trips.                                                                                      |
| Test-DocxProtection /<br>Test-XlsxProtection | Inspect an OpenXML file's internal XML (word/settings.xml, xl/workbook.xml) for write-protection / encryption markers.                                                                                                                             |
| Test-Ppt2003HasOpenPassword                  | Fallback that opens a .ppt read-only via COM to confirm whether it's password-to-open.                                                                                                                                                             |
| IsOfficeFilePasswordProtected                | Older/simple header-based password check (largely superseded by the detailed detectors above).                                                                                                                                                     |

### Property writing (the actual tagging)

| Function                    | Purpose                                                                                                                                                                                                                                                                                                       |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Set-OpenXmlProperties       | Writes custom properties into a <b>modern OpenXML</b> file by manipulating the ZIP package directly — <b>no Office required</b> . Rebuilds docProps/custom.xml, and patches [Content_Types].xml and _rels/.rels so the properties are valid. Writes to a temp file and atomically swaps it over the original. |
| Set-OfficeDocCustomProperty | Writes a single custom property into an <b>open COM Document object</b> (legacy Office path), using reflection against CustomDocumentProperties; if the property already exists it deletes and re-adds it.                                                                                                    |

### Parallel batch processors

| Function             | Purpose                                                                                                                                                                                                                     |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Process-OpenXmlBatch | Tags modern Office files in parallel runspace via Set-OpenXmlProperties. Skips locked/encrypted files, computes the age flags, writes properties, restores timestamps, and logs each outcome. Fast — doesn't launch Office. |



| Function         | Purpose                                                                                                                                                                                                                                                                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Process-ComBatch | Tags <b>legacy</b> Office files via <b>Office COM automation</b> (Word/Excel/PowerPoint), one runspace per file up to a concurrency cap. Opens each file hidden with alerts disabled, writes the three properties, saves, quits, and releases COM objects. This is the fragile, resource-heavy path and is why the kill-monitor exists. |
| Process-PdfBatch | Tags PDFs by shelling out to the Python script (update_pdf_properties*.py) once per file, passing the three Name=Value properties, and interpreting the Python <b>return code</b> (see §7).                                                                                                                                             |

### Error handling, logging, and safety

| Function                                              | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Handle-FileProcessingError                            | Central error handler for the COM path. Classifies COM/RPC errors (e.g. 0x800706BA “RPC server unavailable”), decides whether to continue to the next file, cleans up the COM app, restores timestamps, and logs.                                                                                                                                                                                                                                                                                                                                               |
| Add-ContentSafe                                       | Thread-safe append-to-file with retry/back-off — used everywhere for logging, since many parallel runspace write to the same log files.                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Write-Log / Write-LogProcess                          | Structured log writers (per-file messages and a pipe-delimited progress/status record used by the SQL loader).                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Test-FileExists / Test-FileLocked                     | Pre-flight checks (with retries) to skip files that are missing or open/locked before spending effort on them.                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Start-KillProcessMonitor /<br>Stop-KillProcessMonitor | Launch/stop an <b>external watchdog</b> PowerShell process that periodically kills any WINWORD/EXCEL/POWERPNT that has been running longer than a threshold — the safety net against a COM call hanging on a single bad file and stalling the whole run. The monitor <b>self-restarts on a fatal error</b> (up to 10 restarts, 5-second pause between, each restart logged with its count) rather than dying silently, and runs fully unattended — an earlier version's blocking Read-Host was removed so it can never sit waiting for input on a headless run. |
| Invoke-ExecuteTaggingSafely                           | Top-level wrapper: starts the kill-monitor, runs Execute_Tagging, drains any lingering Office processes on completion, stops the monitor, and sets a clean exit code — <b>except</b> it deliberately exits with code <b>42</b> on an RPC failure so the pipeline can detect it and <b>restart</b> the run.                                                                                                                                                                                                                                                      |

### 6.3 What each section should do (detailed)

**Parameter block & guard.** Requires -DrivePath and -ScriptPath; aborts immediately if DrivePath isn't an accessible container. This stops a misconfigured pipeline run from doing nothing silently.

**OLE/format parsing section (Read-OleStream, Test-LegacyOfficeProtection, Test-OfficeEncrypted, Get-OfficeFormat).** These read raw file bytes to understand a file *before* opening it. The intent is defensive: legacy Office files can be encrypted or write-reserved, and blindly opening them via COM would either prompt for a password (hanging a headless run) or silently fail. By parsing the OLE compound-file header, FAT, and directory streams in memory, the script decides up front whether a file is safe to tag or must be skipped. Files loaded into memory once to avoid repeated network round-trips on shares.

**File discovery section (Get-ApplicableFiles).** Recursively enumerates the target tree and applies the **in-scope filter** (extension + age thresholds + size + not a temp file). This is the definition of “which files are candidates,” and the 540-day / 1095-day thresholds here **match** the meaning of the LastAccessed18Months / Created3Years properties written later. Access errors on individual folders are logged and skipped, not fatal.

**Queueing & dispatch section (Update-FileAgeProperties).** Files are classified and buffered into three queues. Each queue has a **batch trigger** (how many files accumulate before dispatch) and a separate **parallel-items** cap (how many runspace run at once within a batch). These are independent controls: batch triggers reduce pool open/close overhead; parallel-items throttles actual concurrency. After each batch it calls Wait-AndCollectJobs to fully tear the pool down before moving on — this is what keeps memory flat over a long run. Any partial queues are drained at the end.

**OpenXML processing (Process-OpenXmlBatch + Set-OpenXmlProperties).** For modern formats, tagging is pure file manipulation: open the file as a ZIP, rebuild the custom-properties part, fix the content-types and relationships parts, write to a temp file, atomically move it into place. No Office process is launched, so this path is fast and low-risk. Timestamps are restored afterwards.

**Legacy COM processing (Process-ComBatch + Set-OfficeDocCustomProperty).** For .doc/.xls/.ppt, the script must launch the actual Office application via COM, open the file hidden with alerts/macros disabled, write properties, save, close, and then **rigorously release** the COM objects and force GC. This is the resource-heavy, failure-prone path. Every error routes through Handle-FileProcessingError, and the whole path is protected by the external kill-monitor so a single stuck document can't wedge the run.

**PDF processing (Process-PdfBatch).** Delegates to the Python helper (see §7), one process per file, reading the return code to record success / encrypted / signed / error. Uses the Python-installed path configured near the top of the function — **verify this path on any new server** (it's a common break point).

**Kill-monitor section (Start-KillProcessMonitor).** Builds a self-contained monitor script, base64-encodes it, and launches it as an independent PowerShell process that loops, killing long-running Office processes and restarting itself on error. The monitor is wrapped in an outer restart loop (up to **10 restarts**, a **5-second** pause between each, with the restart count written to the log every time) so a transient fault in the watchdog itself doesn't silently take the safety net offline for the rest of a long run — genuine per-file errors inside the loop are logged as warnings and don't count as a restart, only a fatal exception does. It also has **no interactive prompts** (an earlier version's Read-Host was removed), which matters because this process is meant to run unattended for the full duration of the pipeline job. This runs alongside the main tagging work and is stopped (after draining Office) when tagging completes.

**Bottom-of-file execution.** Sets \$global:RpcFailureDetected = \$false, calls Invoke-ExecuteTaggingSafely, and if an RPC failure was detected emits ##[error]Restart required due to RPC failure and **exits 42** — the signal the pipeline uses to retry.

**Housekeeping note for the maintainer:** the script contains a couple of **duplicated .ppt handling blocks** and some commented-out test scaffolding at the very bottom. These are historical and don't affect the executing path, but are worth tidying when the script is next revised. Also note the Python path and some backup paths are effectively hard-coded — treat these as configuration to check per-server.

## 7. Technical walkthrough — update\_pdf\_properties.py

Called once per PDF by Process-PdfBatch. Invocation:

```
python update_pdf_properties.py <pdf_file> <Name=Value> [<Name=Value> ...]
```

```
e.g. python update_pdf_properties.py "C:\docs\file.pdf" "OriginalPath=C:\docs\file.pdf"
"LastAccessed18Months=True" "Created3Years=False"
```

**Return codes** (read by PowerShell to decide the outcome):

| Code | Meaning                                                                       |
|------|-------------------------------------------------------------------------------|
| 0    | Success — properties written                                                  |
| 1    | File is <b>encrypted</b> — skipped                                            |
| 2    | File is <b>digitally signed</b> — skipped (tagging would break the signature) |
| -1   | Unexpected error                                                              |

### Functions:

| Function                                    | Purpose                                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| is_signed(path)                             | Reads the PDF's AcroForm SigFlags to detect a digital signature. Signed PDFs are left untouched. Returns whether the PDF is encrypted. Encrypted PDFs are skipped.                                                                                                                                                                                                                                                    |
| is_encrypted(path)                          |                                                                                                                                                                                                                                                                                                                                                                                                                       |
| update_pdf_properties(pdf_file, properties) | The main routine: bails out early if the file is encrypted or signed; optionally backs the original up as a ZIP; copies all pages into a new PdfWriter; merges existing metadata with the new custom properties (each key given a leading / per the PDF spec); writes to a <b>PID-tagged temp file</b> ; then <b>atomically swaps</b> it over the original (os.remove + os.rename) and removes the backup on success. |
| parse_args(argv)                            | Parses the Name=Value arguments into a dictionary; errors clearly if an argument isn't in the right format.                                                                                                                                                                                                                                                                                                           |

**Why the single-call / one-pass design matters:** an earlier version wrote each property in a separate call, which rewrote the whole file three times and — worse — opened a window between os.remove and os.rename on each pass where the file didn't exist, risking file-not-found errors for any concurrent access. Writing **all properties in one read/write pass** eliminated that window and the redundant rewrites. The **PID-tagged temp filename** is what makes it safe to run many of these Python processes concurrently in the same directory without them colliding.

**Note:** the PowerShell references update\_pdf\_properties\_new.py in one place while the project file is update\_pdf\_properties.py. Confirm which filename is actually deployed on each server so the PDF path doesn't silently fail.

## 8. The reporting side (supporting scripts)

Tagging makes the files *actionable*; reporting makes the estate *visible*. These run alongside the tagging process:

| Script                                         | Purpose                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Get-FileListing-Functions-Servers-Parallel.ps1 | The parallel file-listing engine for on-server drives — the architectural template the file-share listing below is built on. Walks one or more server drives in parallel runspaces, streams batched metadata rows to a tab-delimited output, and supports resume.                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Get-FileShareListing-Parallel.ps1              | The <b>Azure file share</b> counterpart to the servers listing (see §8.1). Maps the share, walks it in parallel, and produces a <b>single</b> tab-delimited listing for the whole share, ready for SQL load.                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| UDR-FolderReport.ps1                           | Walks the target drive in parallel and produces CSV reports of folder contents — file counts, subfolder counts, total size, and a breakdown by file type (PDF / legacy-COM / OpenXML / Other). Runs in Depth, Full, or Both mode; resolves mapped drive letters to UNC paths first.                                                                                                                                                                                                                                                                                                                                                                                                    |
| Load-FolderReportsToSql.ps1                    | Bulk-loads the folder-report CSVs into SQL Server, tracking each file by name + SHA256 hash + row count so re-runs skip already-loaded files.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| Load-AddPropertiesStatusToSql.ps1              | Loads the pipe-delimited<br>*AddPropertiesStatus*.txt progress files (produced by the tagging run) into SQL, detecting New / Unchanged / Changed files by hash and reloading only what changed. Now searches <b>recursively</b> (-Recurse) under -ReportsPath, so status files sitting in subfolders are picked up too — previously it only looked directly inside the given folder. (Note: matching is still by filename only, not full path — if status files with identical names ever exist in two different subfolders, the second is treated as already-loaded and skipped. Not expected to occur given the timestamped naming, but worth knowing if the folder layout changes.) |
| Load-CatalogueDataToSql.ps1                    | Stages tab-delimited catalogue extracts into SQL, enriching rows with parsed dates, file-type, and retention-scope flags. Uses size + last-write-time (not content hash) for change detection specifically so it doesn't <i>hydrate</i> online-only<br>SharePoint/OneDrive placeholders just to check them.                                                                                                                                                                                                                                                                                                                                                                            |
| Load-FileShareDataToSql.ps1                    | Companion loader for the Azure file share listing (see §8.1). A sibling of<br>Load-CatalogueDataToSql.ps1 with identical enrichment/staging/change-detection logic, but defaulting to its own SQL tables so the file-share dataset lands separately.                                                                                                                                                                                                                                                                                                                                                                                                                                   |

Together these give a queryable picture in SQL of where the in-scope, out-of-retention files are, which

supports the quarantine decisions.

### 8.1 Azure file share cataloguing (Get-FileShareListing-Parallel.ps1 + Load-FileShareDataToSql.ps1)

Alongside the on-server drives, the estate includes **Azure file shares**, which need cataloguing into the same SQL reporting so their contents are visible for retention/quarantine decisions. This pair does that.

**Get-FileShareListing-Parallel.ps1** rebuilds the sequential file-share listing on the same parallel architecture as **Get-FileListing-Functions-Servers-Parallel.ps1**, so it behaves consistently with the rest of the reporting. Its distinct behaviours are:

- **Share mapping.** It finds a free drive letter (scanning **Z→A**) and maps the Azure file share (`\\{{storage account}}.file.core.windows.net\{{share name}}`) with `New-PSDrive`, walks the share via that temporary letter, and **translates every stored ContainingPath back to the share's UNC path** so the output doesn't depend on which letter happened to be free. The drive is **always unmapped** on exit (success, failure, or `Ctrl-C`) via a `finally` block.
- **Single output per share.** Unlike the multi-file server listings, it writes **one** fixed-name, tab-delimited output for the whole share (no timestamp), so a resumed run appends to the same file. The progress log *is* timestamped so each run/resume gets a clean log.
- **Parallelism & memory.** Runspace-pool parallelism (`-ParallelItems`) with streamed batch flushing (`-BatchSize`) — it never holds the whole file list in memory.
- **Snapshot exclusion.** It skips any `~snapshot` folder (Azure Files exposes **share snapshots** under that name, which would otherwise be walked and double-count files) as well as Office `~$` lock files.
- **Resume.** R/O/X resume detection (`Resume / Overwrite / eXit` at the prompt): a resumed run reloads already-processed paths and appends only new ones. *(If a previous listing was produced with malformed drive-letter paths, choose **Overwrite** rather than Resume — a resume keeps the old rows and only fixes newly-added ones.)*
- **Credentials.** Takes an optional `-Credential`. Omit it to use the ambient identity (the `{{windows domain}}\UDRTagging account`); for a **storage-account-key** mount, supply a `PSCredential` whose username is `AZURE\{{storage account}}` (or `localhost\{{storage account}}`) and whose password is the storage account key.

**Load-FileShareDataToSql.ps1** is a sibling of **Load-CatalogueDataToSql.ps1**, following the project's one-loader-per-dataset convention. It is functionally identical — same server/database, same enrichment SQL and columns, same size+last-write-time change detection, staging, batched insert, and load-history — **except** its target/staging/history tables default to the file-share dataset's own tables (e.g. `dbo.fileshare_data*`) and its `-SourceFolder` points at the `FileShare` subfolder the listing script writes to. Point both at the same table names if you ever want the file-share and catalogue datasets merged.

#### Operational notes for this pair:

- **Working-path convention is shared between the two scripts.** The listing writes to `<WorkingPath>\FileShare\<share>_FileShareListing.txt`; the loader's `-SourceFolder` defaults to the matching `...\FileListing\FileShare` folder. If you change `-WorkingPath` on the writer, update `-SourceFolder` on the loader to match, or the load finds nothing.
- **Re-sign after any edit.** Like every script here, it runs under `AllSigned`, so run the appropriate `Sign - *` step over it before running — any edit invalidates the previous signature (see §4).
- **Placeholders.** The storage-account FQDN, share name, and working-path server have been shown as placeholders (`{{storage account}}`, `{{share name}}`) for security — substitute the real values in your controlled copy of the scripts/pipeline.

## 8.2 Catalogue loader — SharePoint/OneDrive-aware evolution (Stage-CatalogueData.ps1)

A further-developed variant of Load-CatalogueDataToSql.ps1, built to be safe against **sync'd SharePoint/OneDrive folders**, where files can appear as **online-only placeholders** that haven't actually been downloaded to disk. Same core job (stage → enrich → insert, with a load-history table so unchanged files are skipped) but two points evolved beyond the base loader and are worth knowing if you're comparing the two or deciding which is authoritative in your deployment:

- **Change detection was deliberately moved away from a content hash.** An earlier iteration used Get-FileHash (SHA256), but that requires reading the file's actual bytes — which **forces a placeholder to hydrate (download)** even when it turns out to be unchanged, silently pulling every file in the folder down over the network just to check it. It was replaced with the same **size + Last-WriteTimeUtc** comparison the base loader uses, read straight from Get-ChildItem's placeholder metadata, so an unchanged file is never touched. The load-history table's FileHash column is kept (nullable) for backward compatibility with any history table built under the old version, rather than dropped.
- **New -DehydrateAfterProcessing switch.** After a file has been loaded (or confirmed unchanged), this optionally converts it back to an online-only placeholder, so a full run doesn't leave the entire synced folder pinned to local disk. It's a no-op on the skip path (skipped files are never hydrated to begin with) and mainly matters for files that were actually copied and loaded this run.

**Confirm which loader is in production before treating this as authoritative.** This document lists Load-CatalogueDataToSql.ps1 in the reporting table above as the established loader; Stage-CatalogueData.ps1 reflects a later refinement of the same logic. If both exist in your environment, check with whoever actioned this change which one the pipeline currently calls, and update this document to name the correct one going forward.

## 8.3 Running loader scripts unattended (Windows Task Scheduler)

Where a loader (e.g. Load-AddPropertiesStatusToSql.ps1) needs to run on a schedule outside the ADO pipeline — for example, a periodic refresh independent of a tagging run — it can be configured as a **Windows Scheduled Task**. A bare powershell.exe + relative-path setup is fragile under Task Scheduler because the task runs in a different security context with no profile; use this configuration instead:

- **Program/script:** C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
- **Arguments:** -NoProfile -NonInteractive -ExecutionPolicy Bypass -File "C:\full\path\to\Load-AddPropertiesStatusToSql.ps1" -ReportsPath "<path>" -SqlServer "<server>" -database "unstrdata"
- **Start in:** the folder containing the script

Why each switch matters:

| Switch                                 | Reason                                                                                                                                                                                                      |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -File with a <b>full absolute path</b> | Task Scheduler doesn't reliably honour "Start in" depending on how the task is launched — a bare relative path can silently fail to resolve.                                                                |
| -ExecutionPolicy Bypass                | Scheduled tasks run under a different security context than an interactive session; if that context's effective policy is Restricted or AllSigned (see §4), the script can fail to run at all without this. |
| -NoProfile                             | Skips PowerShell profile scripts, avoiding failures from a profile referencing modules/paths that don't exist in the scheduled-task context.                                                                |

| Switch          | Reason                                                                                                                                                                               |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -NonInteractive | Stops PowerShell hanging indefinitely if anything unexpectedly prompts (e.g. a credential dialog) — without a console to respond to, the task would otherwise sit “running” forever. |

Other configuration to set:

- **Run as account:** the account running the task needs read access to -ReportsPath (if it's a UNC path — a mapped drive letter set up under an interactive login won't exist in the scheduled task's session) and the appropriate SQL Server login/permissions on the target database.
- **“Run whether user is logged on or not”** — needed if the task must run unattended with nobody logged in; pair with “Run with highest privileges” if elevation is required.
- **A timeout setting** (“Stop the task if it runs longer than...”) as a safety net, and a decision on whether to retry on failure.
- **Logging:** if the script doesn't already log internally, redirect output for troubleshooting (Task Scheduler's own stdout capture is limited).

## 9. Operational checklist for a new owner

- **Servers:** confirm Office, Python (+pypdf), and the ADO agent are installed on each execution server, and that the tagging/extract account `{{windows domain}}\UDRTagging` has file access, log-write access, and rights to launch COM/Office.
- **Target-drive access:** for every drive/share brought into scope, add `{{windows domain}}\UDRTagging` to the **Local Administrators group of the server hosting that drive** — without it, files under restrictive ACLs are silently skipped and won't be tagged or reported.
- **Agent startup:** always (re)start the agent via `start-agent-detached.ps1` from an interactive session — never run `.cmd` in a foreground window. After starting, check the launch log confirms the agent is **not** a child of the launching shell.
- **Signing:** after any change to a script, ensure the matching `Sign-*.ps1` step runs and reports `Status = Valid`. Watch the **certificate expiry** date.
- **Paths:** verify the Python executable path and the PDF helper filename on each server; verify `Script-Path / DrivePath` pipeline variables use **UNC**, not user-session drive mappings.
- **Timeouts:** the ADO job timeout must be set generously (the default 60 minutes is too short for large shares); the tagging script's resume logic means an interrupted run continues, but a mid-run kill still leaves partial state.
- **Failure signals:** exit code **42** = RPC failure, intended to trigger a pipeline retry. “Agent not contactable” after detachment is confirmed → check for orphaned `WINWORD/EXCEL/POWERPNT` processes and trend handle/memory usage.
- **Azure file share cataloguing:** when running `Get-FileShareListing-Parallel.ps1`, keep its `-WorkingPath` and the loader's `-SourceFolder` in sync, ensure a free drive letter is available for the New-PSDrive mount, and supply `-Credential` (username `AZURE\{{storage account}}`, password = storage key) only if the ambient `{{windows domain}}\UDRTagging` identity can't reach the share. If a prior listing has bad drive-letter paths, **Overwrite** rather than **Resume**.
- **Invisibility:** the whole point is that tagging doesn't disturb the business — confirm timestamps and read-only flags are being restored (they're logged), so tagged files don't suddenly look recently modified.

*End of handover.*